

Conteúdo

1 Basics	1
1.1 basic	1
1.2 Subconjuntos	1
1.3 BinarySearch	1
1.4 CordCompression	1
2 Data Structures	2
2.1 SegTree	2
2.2 Lichao	2
2.3 SegTreeLazy	2
2.4 BIT	3
2.5 chtDinamico	3
2.6 MergeSort Tree	3
2.7 Cht	4
2.8 PrefixSum2D	4
3 Geometry	5
3.1 Poligons	5
3.2 ClosestPair	5
3.3 Ch	5
3.4 Geometry - General	6
3.5 Point	8
3.6 Segment	8
4 Graphs	8
4.1 Bridges	8
4.2 Floyd-Warshall	8
4.3 Toposort	8
4.4 Kosaraju	9
4.5 EulerPath	9
4.6 BellmanFord	10
4.7 Dijkstra	10
4.8 BellmanFordPrintCycle	10
4.9 Prim	10
4.10 BFS	11
4.11 DFS	11
4.12 DSU	11
4.13 MinCostMaxFlow	11
4.14 EulerTour	12
4.15 MinCostMaxFlowSimple	12
4.16 LCA	12
4.17 Kruskal	13
5 dp	13
5.1 editDistance	13
5.2 LIS	13
5.3 subsetSum	13
5.4 LCS	13
5.5 SumOverSubsetDP	14
5.6 knapsack	14
6 Strings	14
6.1 Manacher	14
6.2 hash	14
6.3 trie	15
6.4 KMP	15
7 Math	15
7.1 totient	15
7.2 sieve	15
7.3 Kadane	16
7.4 trianguloPascal	16

7.5 divisors	16
7.6 ModComb	16
8 Theorems	16
8.1 Propriedades Matemáticas	16
9 Extra	18
9.1 Hash Function	18

1 Basics

1.1 basic

STL find() retorna iterador se achou, se nao achou
retorna container.end()

Complexidade:
-> Set, map: $O(\log N)$
-> List, vector, deque, arrays: $O(n)$
-> Unordered maps without collisions: $O(1)$

STL lower_bound() e upper_bound()
Precisa sortar antes
Lower retorna primeiro maior ou igual a target
Upper retorna maior que o target
Complexidade $O(\log N)$

```
04D vector<int> container(1e3);
335 int target = 0;
A2D auto it = find(container.begin(), container.end(),
    target);

1CE auto it_lower = lower_bound(container.begin(), container
    .end(), target);
42F auto it_upper = upper_bound(container.begin(), container
    .end(), target);
```

1.2 Subconjuntos

Bitmask - Geracao de Subconjuntos
Quando usar: $n \leq 20 \sim 22$

Complexidade: $O(n * 2^n)$

```
E8D int main() {
CAF   int n; vector<int> v(n);
D41
D41   // 1 << n = 2^n subconjuntos
827   for (int mask = 0; mask < (1 << n); mask++) {
4AD     vector<int> subset;
603     for (int i = 0; i < n; i++) {
D41
D41         // se o i-esimo bit esta ligado
CBD         if (mask & (1 << i)) {
EAE             subset.push_back(v[i]);
7CB         }
F90     }
D41
D41     // imprime o subconjunto atual
D41     for (int x : subset) {
D54
```

```
16   cout << x << " ";
16   }
199   cout << "\n";
D3D }
17F }
```

1.3 BinarySearch

Busca binaria sem overflow
-> $l + (r - l) / 2$

Complexidade:
-> $O(\log n)$

```
D11 void bs() {
990   vector<int> v;
1C5   long long l = 0;
97C   long long r = LLONG_MAX;
D1F   int target;
D41
3D5   while(l <= r) {
CAD     long long m = l + (r - l) / 2;
D41
CAA     if(v[m] == target) {
C2B         break;
27B     } else if(v[m] < target) {
16D         l = m + 1;
496     } else {
132         r = m - 1;
959     }
BE0   }
02D }
```

1.4 CordCompression

Compressao de coordenadas

Objetivo:
-> transformar valores grandes
em indices pequenos

Complexidade:
-> $O(n \log n)$

```
E84 vector<int> compress(vector<int> &v) {
BB2   vector<int> comp = v;
90E   sort(comp.begin(), comp.end());
C91   comp.erase(unique(comp.begin(), comp.end(), comp.
end());
DAB   vector<int> ans;
D41
1E9   for(int x : v) {
8E9       ans.push_back(
270         lower_bound(comp.begin(), comp.end(), x) -
comp.begin()
616       );
59C   }
D41
BA7   return ans;
```

```

98C }

E8D int main() {
8DE     vector<int> v = {100, 500, 100, 1000, -20};
4D0     vector<int> c = compress(v);
D41
F14     for(int i = 0; i < v.size(); i++) {
A4E         cout << v[i] << " -> " << c[i] << "\n";
51B     }
CA8 }

```

2 Data Structures

2.1 SegTree

```

D41 /* Segment Tree (Point Update, Range Query)
FED  * Complexidade: O(log N) para update e query.
B5D  * Memoria: O(4*N)
C7E  * Requisitos:
07B  * - NODE deve ter: static merge(L, R), apply(V), e
    construtor identidade.
C4C */

D41 /* --- Exemplo de NODE (Soma com Update de Substituicao)
    ---
BF2 struct Node {
F96     ll val = 0;
702     Node(ll v = 0) : val(v) {}
D41
B67     static inline Node merge(const Node& l, const Node&
    r) {
A4B         return Node(l.val + r.val);
2B0     }
D41
D41     // Para Point Set: val = v;
D41     // Para Point Add: val += v;
E84     inline void apply(int v) {
250         val = v;
AF0     }
876 };
C4C */

8A0 template<typename NODE>
383 struct SegTree {
060     int N;
B1C     vector<NODE> seg;
D41
E7A     SegTree(int n) : N(n), seg(4 * n) {}
D41
67A     template<typename T>
373     SegTree(const vector<T>& v) : N(v.size()), seg(4 * v
    .size()) {
231         build(1, 0, N - 1, v);
EF1     }
D41
67A     template<typename T>
5B9     void build(int no, int l, int r, const vector<T>& v)
    {
893         if (l == r) {
C3E             seg[no] = NODE(v[l]);
505             return;
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);

```

```

C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no <<
    1) | 1]);
349     }
D41
663     void update(int no, int l, int r, int idx, int val)
    {
893         if (l == r) {
D88             seg[no].apply(val);
505             return;
EC9         }
27E         int m = (l + r) >> 1;
B73         if (idx <= m) update(no << 1, l, m, idx, val);
99C         else update((no << 1) | 1, m + 1, r, idx, val);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no <<
    1) | 1]);
358     }
D41
38D     NODE query(int no, int l, int r, int a, int b) {
2E6         if (b < l || r < a) return NODE();
83F         if (a <= l && r <= b) return seg[no];
27E         int m = (l + r) >> 1;
AFE         return NODE::merge(query(no << 1, l, m, a, b),
074             query((no << 1) | 1, m + 1, r
    , a, b));
365     }
D41
9B0     void update(int idx, int val) { update(1, 0, N - 1,
    idx, val); }
36F     NODE query(int l, int r) { return query(1, 0, N - 1,
    l, r); }
45A };

```

2.2 Lichao

Li-Chao Tree
Adiciona retas (ax+b) e computa o minimo entre as
retas em um dado x
Cuidado com overflow, se tiver tenta comprimir o x ou
usar convex hull trick

O(log(MA - MI)) de tempo
O(n) memoria

```

5B0 template<ll MI = 11(-1e9), ll MA = 11(1e9)> struct
    lichao {
B3A     struct line {
12D         ll a, b;
CEF         array<int, 2> ch;
FDF         line(ll a_ = 0, ll b_ = LINF) :
423             a(a_), b(b_), ch({-1, -1}) {}
888         ll operator () (ll x) { return a*x + b; }
D1D     };
17B     vector<line> ln;
D41
DF8     int ch(int p, int d) {
E85         if (ln[p].ch[d] == -1) {
9AF             ln[p].ch[d] = ln.size();
CDC             ln.emplace_back();
BC1         }
EF2         return ln[p].ch[d];
86A     }
021     lichao() { ln.emplace_back(); }
D41
C33     void add(line s, ll l=MI, ll r=MA, int p=0) {
3E3         ll m = (l+r)/2;

```

```

911     bool L = s(l) < ln[p](l);
D37     bool M = s(m) < ln[p](m);
03B     bool R = s(r) < ln[p](r);
825     if (M) swap(ln[p], s), swap(ln[p].ch, s.ch);
CAC     if (s.b == LINF) return;
F6D     if (L != M) add(s, l, m-1, ch(p, 0));
898     else if (R != M) add(s, m+1, r, ch(p, 1));
76E     }
092     ll query(int x, ll l=MI, ll r=MA, int p=0) {
11B         ll m = (l+r)/2, ret = ln[p](x);
9DB         if (ret == LINF) return ret;
529         if (x < m) return min(ret, query(x, l, m-1, ch(p, 0)
    ));
81A         return min(ret, query(x, m+1, r, ch(p, 1)));
FBA     }
59B };

```

2.3 SegTreeLazy

```

D41 /* Lazy Segment Tree (Range Update, Range Query)
E0D  * Realiza atualizacoes e consultas em range.
FED  * Complexidade: O(log N) para update e query.
B5D  * Memoria: O(4*N)
C7E  * Requisitos:
3F4  * - NODE deve ter: static merge(L, R), apply(TAG, L, R)
    e
BF2  * construtor identidade.
3A3  * - TAG deve ter: compose(TAG) e construtor identidade.
C4C */

D41 /* --- Exemplo de NODE e TAG (Soma e Multiplicacao com
    Modulo) ---
7E5 struct Tag {
B7F     ll mul = 1, add = 0;
611     void inline compose(const Tag& t) {
786         add = (add * t.mul + t.add) % MOD;
81E         mul = (mul * t.mul) % MOD;
A7E     }
120 };

BF2 struct Node {
F96     ll val = 0;
702     Node(ll v = 0) : val(v) {}
B67     static inline Node merge(const Node& l, const Node&
    r) {
B19         return Node((l.val + r.val) % MOD);
40E     }
EE5     void inline apply(const Tag& t, int l, int r) {
D8C         val = (val * t.mul + t.add * (r - l + 1)) % MOD;
385     }
EB3 };
C4C */

708 template<typename NODE, typename TAG>
2BA struct LazySegmentTree {
060     int N;
B1C     vector<NODE> seg;
71A     vector<TAG> lazy;
D41
F0F     LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 *
    n) {}
D41
67A     template<typename T>
1BA     LazySegmentTree(const vector<T>& v)
9EB         : N(v.size()), seg(4 * v.size()), lazy(4 * v
    .size()) {}

```

```

231     build(1, 0, N - 1, v);
5EE }
D41
67A template<typename T>
5B9 void build(int no, int l, int r, const vector<T>& v)
{
893     if (l == r) {
C3E         seg[no] = NODE(v[l]);
505         return;
B2E     }
27E     int m = (l + r) >> 1;
35E     build(no << 1, l, m, v);
C55     build((no << 1) | 1, m + 1, r, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no <<
1) | 1]);
349 }
D41
void push(int no, int l, int r) {
27E     int m = (l + r) >> 1;
1EA     int e = no << 1, d = e | 1;
D41
25F     seg[e].apply(lazy[no], l, m);
115     lazy[e].compose(lazy[no]);
D41
4E1     seg[d].apply(lazy[no], m + 1, r);
7CC     lazy[d].compose(lazy[no]);
D41
47F     lazy[no] = TAG();
6A3 }
D41
130 void update(int no, int l, int r, int a, int b,
const TAG& v) {
2E6     if (b < l || r < a) return;
02B     if (a <= l && r <= b) {
8C7         seg[no].apply(v, l, r);
92C         lazy[no].compose(v);
505         return;
81F     }
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
9AB     update(no << 1, l, m, a, b, v);
087     update((no << 1) | 1, m + 1, r, a, b, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no <<
1) | 1]);
A1A }
D41
38D NODE query(int no, int l, int r, int a, int b) {
2E6     if (b < l || r < a) return NODE();
83F     if (a <= l && r <= b) return seg[no];
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
AFE     return NODE::merge(query(no << 1, l, m, a, b),
074         query((no << 1) | 1, m + 1, r,
, a, b));
80A }
D41
0D7 void update(int l, int r, const TAG& v) { update(1,
0, N - 1, l, r, v); }
36F NODE query(int l, int r) { return query(1, 0, N - 1,
l, r); }
54E };

```

2.4 BIT

BIT - Fenwick Tree

Complexidade:
- Build: O(n)
- Single Update: O(log n)
- Query: O(log n)

```

714 struct BIT {
116     vector<int> bit;
060     int N;
D41
FE2     BIT(){}
D41
118     BIT(const vector<int>& a) {
A6F         N = a.size();
FED         bit.assign(N + 1, 0);
D41
929         for (int i = 1; i <= N; ++i)
4A1             bit[i] = a[i - 1];
D41
62B         for (int i = 1; i <= N; ++i) {
EDF             int j = i + (i & -i);
50F             if (j <= N)
985                 bit[j] += bit[i];
55F         }
E30     }
D41
A69     void update(int pos, int val){
685         for(; pos < N; pos += pos & (-pos))
9D0             bit[pos] += val;
35A     }
D41
4D6     int query(int pos){
A93         int sum = 0;
5D0         for(; pos > 0; pos -= pos & (-pos))
6ED             sum += bit[pos];
E66         return sum;
FF7     }
89A };

```

2.5 chtDinamico

Convex Hull Trick Dinamico

Para double, use LINF = 1/.0, div (a,b) = a/b

update(x) atualiza o ponto de intersecao da reta x
overlap(x) verifica se a reta x sobrepoe a proxima
add(a,b) adiciona reta da forma ax + b
query(x) computa maximo de ax + b para entre as retas

O(log(n)) amortizado por insercao
O(log(n)) por query

Cuidado com overflow

```

72C struct Line {
073     mutable ll a, b, p;
8E3     bool operator<(const Line& o) const { return a < o.a;
}
ABF     bool operator<(ll x) const { return p < x; }
469 };

```

```

326 struct dynamic_hull : multiset<Line, less<>> {
33A     ll div(ll a, ll b) {
A20         return a / b - ((a ^ b) < 0 and a % b);
A8A     }
D41
void update(iterator x) {
BBB     if (next(x) == end()) x->p = LINF;
B2A     else if (x->a == next(x)->a) x->p = x->b >= next(x)
->b ? LINF : -LINF;
424     else x->p = div(next(x)->b - x->b, x->a - next(x)->a
);
0C4 }
D41
bool overlap(iterator x) {
F18     update(x);
CFA     if (next(x) == end()) return 0;
A4A     if (x->a == next(x)->a) return x->b >= next(x)->b;
D40     return x->p >= next(x)->p;
901 }
D41
void add(ll a, ll b) {
176     auto x = insert({a, b, 0});
1C7     while (overlap(x)) erase(next(x)), update(x);
4AB     if (x != begin() and !overlap(prev(x))) x = prev(x),
DBC     update(x);
0FC     while (x != begin() and overlap(prev(x)))
4D2         x = prev(x), erase(next(x)), update(x);
48F }
D41
ll query(ll x) {
4AD     assert(!empty());
229     auto l = *lower_bound(x);
D41
ABA     return l.a * x + l.b;
3F5 }
8F2 };

```

2.6 MergeSortTree

MergeSort Tree

Se for construída sobre um array:

count(i, j, a, b) retorna quantos elementos de v[i..j] pertencem a [a, b]
report(i, j, a, b) retorna os índices dos elementos de v[i..j] que pertencem a [a, b]
retorna o vetor ordenado

Se for construída sobre pontos (x, y):

count(x1, x2, y1, y2) retorna quantos pontos pertencem ao retângulo (x1, y1), (x2, y2)
report(x1, x2, y1, y2) retorna os índices dos pontos que pertencem ao retângulo (x1, y1), (x2, y2)
retorna os pontos ordenados lexicograficamente (assume x1 ≤ x2, y1 ≤ y2)

kth(y1, y2, k) retorna o índice do ponto com k-ésimo menor x dentre os pontos que possuem y em [y1, y2] (0 based)

Se quiser usar para achar k-ésimo valor em range, construir com ms_tree t(v, true), e chamar kth(1, r, k)

Usa O(n log(n)) de memória

Complexidades:
construir - O(n log(n))
count - O(log(n))
report - O(log(n) + k) para k índices retornados
kth - O(log(n))

```
C6C template <typename T = int> struct ms_tree {
6F7     vector<tuple<T, T, int>>> v;
1A8     int n;
5EE     vector<vector<tuple<T, T, int>>> t; // {y, idx, left}
6AE     vector<T> vy;
D41
78C     ms_tree(vector<pair<T, T>>& vv) : n(vv.size()), t(4*n)
, vy(n) {
E80         for (int i = 0; i < n; i++) v.push_back({vv[i].
first, vv[i].second, i});
FCA         sort(v.begin(), v.end());
224         build(1, 0, n-1);
01A         for (int i = 0; i < n; i++) vy[i] = get<0>(t[1][i
+1]);
45E     }
DAC     ms_tree(vector<T>& vv, bool inv = false) { // inv:
inverte indice e valor
8E8         vector<pair<T, T>> v2;
E1E         for (int i = 0; i < vv.size(); i++)
196             inv ? v2.push_back({vv[i], i}) : v2.push_back({i
, vv[i]});
CCA         *this = ms_tree(v2);
F23     }
2C6     void build(int p, int l, int r) {
1D2         t[p].push_back({get<0>(v[l]), get<0>(v[r]), 0}); //
{min_x, max_x, 0}
5C8         if (l == r) return t[p].push_back({get<1>(v[l]), get
<2>(v[l]), 0});
EE4         int m = (l+r)/2;
BD9         build(2*p, l, m), build(2*p+1, m+1, r);
D41
32D         int L = 0, R = 0;
A03         while (t[p].size() <= r-l+1) {
68E             int left = get<2>(t[p].back());
4AA             if (L > m-l or (R+m+1 <= r and t[2*p+1][l+R] < t
[2*p][l+L])) {
8CF                 t[p].push_back(t[2*p+1][l + R+1]);
DA0                 get<2>(t[p].back()) = left;
```

```
5E2         continue;
CE0     }
249     t[p].push_back(t[2*p][l + L+1]);
339     get<2>(t[p].back()) = left+1;
208 }
2EB }
D41
DD3     int get_l(T y) { return lower_bound(vy.begin(), vy.end
(), y) - vy.begin(); }
EBB     int get_r(T y) { return upper_bound(vy.begin(), vy.end
(), y) - vy.begin(); }
D41
F62     int count(T x1, T x2, T y1, T y2) {
902         function<int(int, int, int)> dfs = [&](int p,
int l, int r) {
7C6             if (l == r or x2 < get<0>(t[p][0]) or get
<1>(t[p][0]) < x1) return 0;
2BB             if (x1 <= get<0>(t[p][0]) and get<1>(t[p][0]) <=
x2) return r-l;
784             int nl = get<2>(t[p][1]), nr = get<2>(t[p][r]);
EB6             return dfs(2*p, nl, nr) + dfs(2*p+1, l-nl, r-nr);
122         };
7CB         return dfs(1, get_l(y1), get_r(y2));
F65     }
002     vector<int> report(T x1, T x2, T y1, T y2) {
4B8         vector<int> ret;
85E         function<void(int, int, int)> dfs = [&](int p, int l
, int r) {
882             if (l == r or x2 < get<0>(t[p][0]) or get
<1>(t[p][0]) < x1) return;
8DA             if (x1 <= get<0>(t[p][0]) and get<1>(t[p][0]) <=
x2) {
E00                 for (int i = l; i < r; i++) ret.
push_back(get<1>(t[p][i+1]));
305                 return;
067             }
784             int nl = get<2>(t[p][1]), nr = get<2>(t[p][r]);
194             dfs(2*p, nl, nr), dfs(2*p+1, l-nl, r-nr);
12B         };
8AD         dfs(1, get_l(y1), get_r(y2));
EDF         return ret;
668     }
985     int kth(T y1, T y2, int k) {
902         function<int(int, int, int)> dfs = [&](int p,
int l, int r) {
150             if (k >= r-l) {
941                 k -= r-l;
DAA                 return -1;
B8D             }
8DA             if (r-l == 1) return get<1>(t[p][l+1]);
784             int nl = get<2>(t[p][1]), nr = get<2>(t[p][r]);
072             int left = dfs(2*p, nl, nr);
3B6             if (left != -1) return left;
04D             return dfs(2*p+1, l-nl, r-nr);
A1B         };
7CB         return dfs(1, get_l(y1), get_r(y2));
635     }
1CE };
```

2.7 Cht

Convex Hull Trick Estático
Adds tem que serem feitos em ordem de slope
Queries tem que ser feitas em ordem de x

Add O(1) amortizado
Get O(1) amortizado

```
D41 // Convex Hull Trick Estático
D41 //
D41 // adds tem que serem feitos em ordem de slope
D41 // queries tem que ser feitas em ordem de x
D41 //
D41 // add O(1) amortizado, get O(1) amortizado
4B5 struct CHT {
942     int it;
AC1     vector<ll> a, b;
45E     CHT():it(0){}
0BB     ll eval(int i, ll x){
93D         return a[i]*x + b[i];
B2A     }
63A     bool useless(){
A20         int sz = a.size();
35F         int r = sz-1, m = sz-2, l = sz-3;
2CA         #warning cuidado com overflow!
D71         return (b[l] - b[r])*(a[m] - a[l]) <
413             (b[l] - b[m])*(a[r] - a[l]);
948     }
BF4     void add(ll A, ll B){
7F5         a.push_back(A); b.push_back(B);
565         while (!a.empty()){
233             if ((a.size() < 3) || !useless()) break;
ECB             a.erase(a.end() - 2);
568             b.erase(b.end() - 2);
B21         }
D27         it = min(it, int(a.size()) - 1);
6DF     }
81B     ll get(ll x){
46A         while (it+1 < a.size()){
3C4             if (eval(it+1, x) > eval(it, x)) it++;
F97             else break;
FE9         }
420         return eval(it, x);
B44     }
A91 };
```

2.8 PrefixSum2D

Code by SamuelH12
Complexidade:
-> Calculer: O(N^2)
-> Queries: O(1)

```
3CE const int MAXN = 1e3 + 5;
B77 int ps [MAXN][MAXN];
131 void calcPS2d(){
998     for (int i = 1; i < MAXN; i++) ps[0][i] += ps[0][i -
1]; //inicializo a 1a linha
003     for (int i = 1; i < MAXN; i++) ps[i][0] += ps[i -
1][0]; //inicializo a 1a coluna
D41
7E4     for (int i = 1; i < MAXN; i++)
582         for (int j = 1; j < MAXN; j++)
0B7             ps[i][j] += ps[i - 1][j] + ps[i][j - 1] - ps[i -
1][j - 1];
```

```

577 }
E68 int queryPS2d(int xi, int yi, int xf, int yf){ return ps
    [xf][yf] - ps[xf][yi-1] - ps[xi-1][yf] + ps[xi-1][yi
    -1]; }

```

3 Geometry

3.1 Polygons

01A #

Returns **twice area of a simple polygon**. area*2 (Shoelace Formula: signed cross product sum)

```

5AB ll Area2x(vector<PT>& p){
604 ll area = 0;
37F for(int i=2; i < p.size(); i++){
20A area += (p[i]-p[0]) % (p[i-1]-p[0]);
199 return abs(area);
64B }

```

Returns if a point is **inside a triangle** (or in the border).

```

5CA bool ptInsideTriangle(PT p, PT a, PT b, PT c){
58B if((b-a) % (c-b) < 0) swap(a, b);
805 if(onSegment(a,b,p)) return 1;
1A3 if(onSegment(b,c,p)) return 1;
1DB if(onSegment(c,a,p)) return 1;
13A bool x = (b-a) % (p-b) < 0;
B85 bool y = (c-b) % (p-c) < 0;
CE5 bool z = (a-c) % (p-a) < 0;
4B5 return x == y && y == z;
9C6 }

```

Returns the **center of mass** for a polygon. O(n)

```

303 PT polygonCenter(const vector<PT>& v){
313 PT res(0, 0); double A = 0;
E3C for(int i=0, j=v.size()-1; i<v.size(); j=i++){
FF1 res = res + (v[i]+v[j]) * (v[j]%v[i]);
587 A += v[j] % v[i];
D4F }
33C return res / A / 3;
CD0 }

```

PolygonCut: Returns the vertices of the polygon cut away at the left of the line s->e.polygonCut(p, PT(0,0), PT(1,0));



```

767 vector<PT> polygonCut(const vector<PT>& poly, PT s, PT e
){
81A vector<PT> res;
6F1 for(int i=0; i<poly.size(); i++){
431 PT cur = poly[i], prev = i ? poly[i-1] : poly.back()
;
C5F auto a = s.cross(e, cur), b = s.cross(e, prev);
498 if((a < 0) != (b < 0)) res.push_back(cur + (prev -
cur) * (a / (a - b)));
DDB if(a < 0) res.push_back(cur);
1E0 }
B50 return res;
D6D }

```

Pick's theorem for **lattice points** in a simple polygon. (lattice points = integer points)Area = insidePts + boundPts/2 - 12A - b + 2 = 2i

```

CDC ll cntInsidePts(ll area_db, ll bound){ return (area_db +
2LL - bound)/2; }
ED9 ll latticePointsInSeg(PT a, PT b){
FA7 ll dx = abs(a.x - b.x);
97A ll dy = abs(a.y - b.y);
695 return gcd(dx, dy) + 1;
FA7 }

```

3.2 ClosestPair

Closest pairs - Par de pontos que tem a menor distancia Euclidiana entre si
O(n log n)

```

531 pii ClosestPair(vector<PT<ll>>& pts) {
62D ll dist = (pts[0]-pts[1]).dist2();
1A2 pii ans(0, 1);
CA0 int n = pts.size();
47B vector<int> p(n);
469 iota(begin(p),end(p),0);
F85 sort(p.begin(), p.end(), [&](int a, int b) { return
pts[a].x < pts[b].x; });
3E7 set<pii> points;
526 auto sqr = [](long long x) -> long long { return x *
x; };
637 for(int l = 0, r = 0; r < n; r++) {
39E while(sqr(pts[p[r]].x - pts[p[l]].x) > dist) {
9F7 points.erase(pii(pts[p[l]].y, p[l]));
63B l++;
FFB }
7CF ll delta = sqrt(dist) + 1;
92B auto itl = points.lower_bound(pii(pts[p[r]].y -
delta, -1));
6C3 auto itr = points.upper_bound(pii(pts[p[r]].y +
delta, n + 1));
901 for(auto it = itl; it != itr; it++) {
C66 ll curDist = (pts[p[r]] - pts[it->second]).dist2
();
0CF if(curDist < dist) {
1E3 dist = curDist;
8E7 ans = pii(p[r], it->second);
AE0 }
C07 }
EEA points.insert(pii(pts[p[r]].y, p[r]));
9B6 }
EBB if(ans.first > ans.second)
6FE swap(ans.first, ans.second);
BA7 return ans;
D1D }

```

3.3 Ch

Given a vector of points, return the convex hull in CCW order.
A convex hull is the smallest convex polygon that contains all the points.

If you want colinear points in border, change the >=0 to >0 in the while's.

WARNING:if collinear and all input PT are collinear, may have duplicated points (the round trip)

```

CD7 vector<PT> ConvexHull(vector<PT> pts, bool sorted=false)
{
EC1 if(!sorted) sort(begin(pts), end(pts));
6E7 pts.resize(unique(begin(pts), end(pts)) - begin(pts));
64B if(pts.size() <= 1) return pts;
D41
B4E int s=0, n=pts.size();
988 vector<PT> h(2*n+1);
D41
AA9 for(int i=0; i<n; h[s++] = pts[i++])
316 while(s > 1 && (pts[i] - h[s-2]) % (h[s-1] - h[s-2])
>= 0 )
351 s--;
D41
61B for(int i=n-2, t=s; ~i; h[s++] = pts[i--])
644 while(s > t && (pts[i] - h[s-2]) % (h[s-1] - h[s-2])
>= 0 )
351 s--;
D41
CBB h.resize(s-1);
81C return h;
CBB } //PT operators needed: {- % == <}

```

Check if a point is inside convex hull (CCW, no collinear).If strict == true, then pt on boundary return falseO(log N)

```

3D7 bool isInside(const vector<PT>& h, PT p, bool strict =
true){
579 int a = 1, b = h.size() - 1, r = !strict;
795 if(h.size() < 3) return r && onSegment(h[0], h.back(),
p);
59E if(h[0].cross(h[a], h[b]) > 0) swap(a, b);
317 if(h[0].cross(h[a], p) >= r || h[0].cross(h[b], p) <=
-r) return false;
48A while(abs(a-b) > 1){
4F7 int c = (a + b) / 2;
142 if(h[0].cross(h[c], p) > 0) b = c;
1B9 else a = c;
7E3 }
B11 return h[a].cross(h[b], p) < r;
EB9 }

```

Check if a point is inside convex hull
O(log N)

```

E13 bool isInside(const vector<PT> &h, PT p){
66D if(h[0].cross(p, h[1]) > 0 || h[0].cross(p, h.back())
< 0) return false;
B28 int n = h.size(), l=1, r = n-1;
E55 while(l != r){
264 int mid = (l+r+1)/2;
B64 if(h[0].cross(p, h[mid]) < 0) l = mid;
943 else r = mid - 1;
D3D }
0F2 return h[l].cross(h[(l+1)%n], p) >= 0;
CBC }

```

Given a convex hull h and a point p, returns the indice of h where the dot product is maximized.This code assumes that there are NO 3 colinear points!

```

DD1 int maximizeScalarProduct(const vector<PT> &h, PT v) {
A75 int ans = 0, n = h.size();
F37 if(n < 20){
830 for(int i=0; i<n; i++){
070 if(v*h[ans] < v*h[i])
C46 ans = i;
BA7 return ans;
E80 }

```



```

D41
866 for(int rep=0; rep<2; rep++){
D47     int l = 2, r = n-1;
E55     while(l != r){
264         int mid = (l+r+1)/2;
9E8         int f = v*h[mid] >= v*h[mid-1];
D41
FCF         if(rep) f |= v*h[mid-1] < v*h[0];
622         else f &= v*h[mid] >= v*h[0];
D41
109         if(f) l = mid;
943         else r = mid - 1;
9A3     }
48D     if(v*h[ans] < v*h[l]) ans = l;
6A2 }
3D0 if(v*h[ans] < v*h[1]) ans = 1;
BA7 return ans;
E80 }

```

3.4 Geometry - General

PONTO & VETOR

th e radianos
angle calcula o angulo do vetor com o eixo x
sarea calcula area com sinal
col se p, q e r sao colin.
ccw e counter-clockwise (anti-horario)
rotaciona 90 graus

RETA

isvert - se e vertical
isinseg - ponto pertence ao segmento
get_t - ponto intersecao
proj - projecao cartesiana
inter - intersecao de dois segmentos
interseg - se dois segmentos se interceptam
distseg - distancia entre dois segmentos

POLIGONO

cut_polygon -> corta poligono com a reta r O(n)
dist_rect -> distancia entre os retangulos a e b (lados paralelos aos eixos), assume que ta representado (inferior esquerdo, superior direito)
pol_area -> area do poligono
inpol -> O(n) retorna 0 se ta fora, 1 se ta no interior e 2 se ta na borda
interpol -> se dois poligonos se intersectam - O(n*m)
distpol -> distancia entre poligonos
convex_hull - O(n log(n)) nao pode ter ponto colinear no convex hull
is_inside -> se o ponto ta dentro do hull - O(log(n))
extreme -> ponto extremo em relacao a cmp(p, q) = p mais extremo q
copiado de <https://github.com/gustavoM32/caderno-zika>

CIRCUNFERENCIA

getcenter -> centro da circunf dado 3 pontos

```

circ_line_inter -> intersecao da circunf (c, r) e
    reta ab
circ_inter -> intersecao da circunf (a, r) e (b, R),
    assume que as retas tem p < q
operator< e == comparador pro set pra fazer sweep
    line com segmentos
    assume que os segmentos tem p < q, comparador pro
    set pra fazer sweep angle com segmentos

```

```

C83 typedef double ld;
E3B const ld DINF = 1e18;
43A const ld pi = acos(-1.0);
107 const ld eps = 1e-9;

```

```
B32 #define sq(x) ((x)*(x))
```

```

D97 bool eq(ld a, ld b) {
BA0     return abs(a - b) <= eps;
BFC }

```

```

B2A struct pt {
C1E     ld x, y;
3DD     pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
5BC     bool operator < (const pt p) const {
059         if (!eq(x, p.x)) return x < p.x;
F98         if (!eq(y, p.y)) return y < p.y;
BB3         return 0;
F61     }
A83     bool operator == (const pt p) const {
ED0         return eq(x, p.x) and eq(y, p.y);
589     }
CB9     pt operator + (const pt p) const { return pt(x+p.x, y+
p.y); }
A24     pt operator - (const pt p) const { return pt(x-p.x, y-
p.y); }
4A8     pt operator * (const ld c) const { return pt(x*c, y*
c); }
A60     pt operator / (const ld c) const { return pt(x/c, y/
c); }
3B6     ld operator * (const pt p) const { return x*p.x + y*p.
y; }
6DF     ld operator ^ (const pt p) const { return x*p.y - y*p.
x; }
5ED     friend istream& operator >> (istream& in, pt& p) {
E37         return in >> p.x >> p.y;
E45     }
A8B };

```

```

B3A struct line {
730     pt p, q;
0D6     line() {}
4B8     line(pt p_, pt q_) : p(p_), q(q_) {}
8D7     friend istream& operator >> (istream& in, line& r) {
4CB         return in >> r.p >> r.q;
858     }
7AB };

```

```

364 ld dist(pt p, pt q) {
5F3     return hypot(p.y - q.y, p.x - q.x);
C68 }

```

```

9D7 ld dist2(pt p, pt q) {
F24     return sq(p.x - q.x) + sq(p.y - q.y);
80F }

```

```

483 ld norm(pt v) {
490     return dist(pt(0, 0), v);
CF7 }

```

```

589 ld angle(pt v) {
587     ld ang = atan2(v.y, v.x);
6F8     if (ang < 0) ang += 2*pi;
19C     return ang;
404 }

```

```

298 ld sarea(pt p, pt q, pt r) {
606     return ((q-p)^(r-q))/2;
1B1 }

```

```

E32 bool col(pt p, pt q, pt r) {
E7D     return eq(sarea(p, q, r), 0);
98C }

```

```

0CD bool ccw(pt p, pt q, pt r) {
FA7     return sarea(p, q, r) > eps;
85D }

```

```

1EF pt rotate(pt p, ld th) {
E5C     return pt(p.x * cos(th) - p.y * sin(th),
FF1         p.x * sin(th) + p.y * cos(th));
41A }

```

```

AB1 pt rotate90(pt p) {
A0D     return pt(-p.y, p.x);
E4A }

```

```

EDC bool isvert(line r) { // se r eh vertical
87D     return eq(r.p.x, r.q.x);
0FB }

```

```

099 bool isinseg(pt p, line r) {
F65     pt a = r.p - p, b = r.q - p;
B04     return eq((a ^ b), 0) and (a * b) < eps;
726 }

```

```

98D ld get_t(pt v, line r) {
6EE     return (r.p^r.q) / ((r.p-r.q)^v);
A0A }

```

```

256 pt proj(pt p, line r) {
BEA     if (r.p == r.q) return r.p;
97A     r.q = r.q - r.p; p = p - r.p;
9F8     pt proj = r.q * ((p*r.q) / (r.q*r.q));
2CD     return proj + r.p;
232 }

```

```

D5C pt inter(line r, line s) {
146     if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return pt(DINF
, DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
111 }

```

```

676 bool interseg(line r, line s) {
19B     if (isinseg(r.p, s) or isinseg(r.q, s)
C21         or isinseg(s.p, r) or isinseg(s.q, r)) return 1;
D41
9FA     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
413         ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
359 }

```

```

FCB ld disttoline(pt p, line r) {
89A     return 2 * abs(sarea(p, r.p, r.q)) / dist(r.p, r.q);
1B7 }

```

```
BCC ld disttoseg(pt p, line r) {
```

```

73D     if ((r.q - r.p)*(p - r.p) < 0) return dist(r.p, p);
951     if ((r.p - r.q)*(p - r.q) < 0) return dist(r.q, p);
A19     return disttoline(p, r);
367 }

11D ld distseg(line a, line b) {
4DF     if (interseg(a, b)) return 0;
D41
349     ld ret = DINF;
341     ret = min(ret, disttoseg(a.p, b));
CEB     ret = min(ret, disttoseg(a.q, b));
093     ret = min(ret, disttoseg(b.p, a));
448     ret = min(ret, disttoseg(b.q, a));
D41
EDF     return ret;
222 }
1A9 vector<pt> cut_polygon(vector<pt> v, line r) {
8AF     vector<pt> ret;
8A4     for (int j = 0; j < v.size(); j++) {
DAC         if (ccw(r.p, r.q, v[j])) ret.push_back(v[j]);
DCE         if (v.size() == 1) continue;
030         line s(v[j], v[(j+1)%v.size()]);
AE3         pt p = inter(r, s);
A3D         if (isinseg(p, s)) ret.push_back(p);
D44     }
8A1     ret.erase(unique(ret.begin(), ret.end()), ret.end());
24D     if (ret.size() > 1 and ret.back() == ret[0]) ret.
pop_back();
EDF     return ret;
253 }

5F5 ld dist_rect(pair<pt, pt> a, pair<pt, pt> b) {
080     ld hor = 0, vert = 0;
34B     if (a.second.x < b.first.x) hor = b.first.x - a.second
.x;
F5F     else if (b.second.x < a.first.x) hor = a.first.x - b.
second.x;
4FD     if (a.second.y < b.first.y) vert = b.first.y - a.
second.y;
80A     else if (b.second.y < a.first.y) vert = a.first.y - b.
second.y;
96F     return dist(pt(0, 0), pt(hor, vert));
630 }

13D ld polarea(vector<pt> v) {
9C5     ld ret = 0;
C6E     for (int i = 0; i < v.size(); i++)
80F         ret += sarea(pt(0, 0), v[i], v[(i + 1) % v.size()]);
D03     return abs(ret);
5DF }

8E7 int inpol(vector<pt>& v, pt p) {
8DE     int qt = 0;
F14     for (int i = 0; i < v.size(); i++) {
BDA         if (p == v[i]) return 2;
6AF         int j = (i+1)%v.size();
E38         if (eq(p.y, v[i].y) and eq(p.y, v[j].y)) {
97F             if ((v[i]-p)*(v[j]-p) < eps) return 2;
5E2             continue;
48B         }
388         bool baixo = v[i].y+eps < p.y;
464         if (baixo == (v[j].y+eps < p.y)) continue;
366         auto t = (p-v[i])^(v[j]-v[i]);
1B4         if (eq(t, 0)) return 2;
839         if (baixo == (t > eps)) qt += baixo ? 1 : -1;
D13     }
B84     return qt != 0;
A64 }

6FF bool interpol(vector<pt> v1, vector<pt> v2) {

```

```

7D1     int n = v1.size(), m = v2.size();
C36     for (int i = 0; i < n; i++) if (inpol(v2, v1[i]))
return 1;
AB8     for (int i = 0; i < n; i++) if (inpol(v1, v2[i]))
return 1;
523     for (int i = 0; i < n; i++) for (int j = 0; j < m; j
++)
0C8         if (interseg(line(v1[i], v1[(i+1)%n]), line(v2[j],
v2[(j+1)%m]))) return 1;
BB3         return 0;
C58 }

494 ld distpol(vector<pt> v1, vector<pt> v2) {
F6B     if (interpol(v1, v2)) return 0;
D41
349     ld ret = DINF;
D41
1C8     for (int i = 0; i < v1.size(); i++) for (int j = 0; j
< v2.size(); j++)
6C2         ret = min(ret, distseg(line(v1[i], v1[(i + 1) % v1.
size()]),
9D9         line(v2[j], v2[(j + 1) % v2.size()])));
EDF     return ret;
125 }

138 vector<pt> convex_hull(vector<pt> v) {
FCA     sort(v.begin(), v.end());
D76     v.erase(unique(v.begin(), v.end()), v.end());
52D     if (v.size() <= 1) return v;
526     vector<pt> l, u;
F14     for (int i = 0; i < v.size(); i++) {
FB2         while (l.size() > 1 and !ccw(l.end()[-2], l.end
()[-1], v[i]))
364             l.pop_back();
C35             l.push_back(v[i]);
58E         }
3E9         for (int i = v.size() - 1; i >= 0; i--) {
F19             while (u.size() > 1 and !ccw(u.end()[-2], u.end
()[-1], v[i]))
7A8                 u.pop_back();
A95                 u.push_back(v[i]);
0B8             }
CFC             l.pop_back(); u.pop_back();
82B             for (pt i : u) l.push_back(i);
792             return 1;
10D         }

483 struct convex_pol {
F50     vector<pt> pol;
D41
D98     convex_pol() {}
A04     convex_pol(vector<pt> v) : pol(convex_hull(v)) {}
D41
8AF     bool is_inside(pt p) {
B6E         if (pol.size() == 0) return false;
EAE         if (pol.size() == 1) return p == pol[0];
67F         int l = 1, r = pol.size();
40C         while (l < r) {
EE4             int m = (l+r)/2;
48F             if (ccw(p, pol[0], pol[m])) l = m+1;
EF3             else r = m;
91C         }
00A         if (l == 1) return isinseg(p, line(pol[0], pol
[1]));
1C0         if (l == pol.size()) return false;
6B0         return !ccw(p, pol[l], pol[l-1]);
D41
719     int extreme(const function<bool(pt, pt)>& cmp) {
B1C         int n = pol.size();

```

```

4A2     auto extr = [&](int i, bool& cur_dir) {
22A         cur_dir = cmp(pol[(i+1)%n], pol[i]);
61A         return !cur_dir and !cmp(pol[(i+n-1)%n], pol
[i]);
364     };
63D     bool last_dir, cur_dir;
A0D     if (extr(0, last_dir)) return 0;
993     int l = 0, r = n;
EAD     while (l+1 < r) {
EE4         int m = (l+r)/2;
F29         if (extr(m, cur_dir)) return m;
44A         bool rel_dir = cmp(pol[m], pol[l]);
B18         if ((!last_dir and cur_dir) or
261             (last_dir == cur_dir and rel_dir == cur_dir)
) {
8A6             l = m;
1F1             last_dir = cur_dir;
94A         } else r = m;
606     }
792     return 1;
56C }
316 int max_dot(pt v) {
EC1     return extreme([&](pt p, pt q) { return p*v > q*
v; });
3B7 }
A54 pair<int, int> tangents(pt p) {
FFB     auto L = [&](pt q, pt r) { return ccw(p, r, q);
};
8FD     auto R = [&](pt q, pt r) { return ccw(p, q, r); };
FA8     return {extreme(L), extreme(R)};
736 }
3EC }

911 pt getcenter(pt a, pt b, pt c) {
174     b = (a + b) / 2;
2AE     c = (a + c) / 2;
98B     return inter(line(b, b + rotate90(a - b)),
3F8     line(c, c + rotate90(a - c)));
A12 }

4B3 vector<pt> circ_line_inter(pt a, pt b, pt c, ld r) {
8AF     vector<pt> ret;
F2B     b = b-a, a = a-c;
4B1     ld A = b*b;
20A     ld B = a*b;
2E9     ld C = a*a - r*r;
1FA     ld D = B*B - A*C;
818     if (D < -eps) return ret;
DC5     ret.push_back(c+a+b*(-B+sqrt(D+eps))/A);
20E     if (D > eps) ret.push_back(c+a+b*(-B-sqrt(D))/A);
EDF     return ret;
CD8 }

AD2 vector<pt> circ_inter(pt a, pt b, ld r, ld R) {
8AF     vector<pt> ret;
B7E     ld d = dist(a, b);
5CE     if (d > r+R or d+min(r, R) < max(r, R)) return ret;
398     ld x = (d*d-R*R+r*r)/(2*d);
183     ld y = sqrt(r*r-x*x);
325     pt v = (b-a)/d;
76E     ret.push_back(a+v*x + rotate90(v)*y);
2CB     if (y > 0) ret.push_back(a+v*x - rotate90(v)*y);
EDF     return ret;
FB1 }

6E0 bool operator <(const line& a, const line& b) {
A13     pt v1 = a.q - a.p, v2 = b.q - b.p;
F82     if (!eq(angle(v1), angle(v2))) return angle(v1) <
angle(v2);

```

```

780     return ccw(a.p, a.q, b.p);
27E }
B14 bool operator==(const line& a, const line& b) {
76C     return !(a < b) and !(b < a);
449 }

2C4 struct cmp_sweepline {
D80     bool operator() (const line& a, const line& b)
        const {
191         if (a.p == b.p) return ccw(a.p, a.q, b.q);
231         if (!eq(a.p.x, a.q.x) and (eq(b.p.x, b.q.x) or a.p.x
+eps < b.p.x))
780             return ccw(a.p, a.q, b.p);
DC0             return ccw(a.p, b.q, b.p);
243         }
367 };

BEF pt dir;
5B0 struct cmp_sweepangle {
D80     bool operator() (const line& a, const line& b)
        const {
522         return get_t(dir, a) + eps < get_t(dir, b);
653     }
97F };

```

3.5 Point

```

len() -> O(sqrt(p*p))
cross() -> (a-p) % (b-p)
quad() -> Cartesian plane quadrant |0+||1-||2--||3+-|
proj() -> projection size from A to B
angle( ) -> Angle between vectors p and q [-pi, pi] |
    acos(a*b/a.len()/b.len())
polarAngle() -> Angle to x-axis [-pi, pi]

```

```

C19 struct PT {
0BE     ll x, y;
0A5     PT(ll x=0, ll y=0) : x(x), y(y) {}
D41
006     PT operator+(const PT&a) const { return PT(x+a.x, y+a.y)
; }
0DC     PT operator-(const PT&a) const { return PT(x-a.x, y-a.y)
; }
954     ll operator*(const PT&a) const { return (x*a.x + y*a.y)
; }
A68     ll operator%(const PT&a) const { return (x*a.y - y*a.x)
; }
B54     PT operator*(ll c) const { return PT(x*c, y*c); }
B25     PT operator/(ll c) const { return PT(x/c, y/c); }
5C7     bool operator==(const PT&a) const { return x == a.x &&
y == a.y; }
B4F     bool operator< (const PT&a) const { return x != a.x ? x
< a.x : y < a.y; }

D41
652     ld len() const { return hypot(x,y); }
3FC     ll cross(const PT&a, const PT&b) const { return (a-*
this) % (b-*this); }
950     int quad() { return (x<0)^3*(y<0); }
94A     bool ccw(PT q, PT r) { return (q-*this) % (r-q) > 0; }
8EA };

33E ld dist(PT p, PT q) { return sqrtl((p-q)*(p-q)); }
0FB ld proj(PT p, PT q) { return p*q / q.len(); }

```

```

C4F const ld PI = acos(-1.0L);
50C ld angle(PT p, PT q) { return atan2(p%q, p*q); }
E07 ld polarAngle(PT p) { return atan2(p.y, p.x); }
AF5 bool cmp_ang(PT p, PT q) { return p.quad() != q.quad() ?
p.quad() < q.quad() : q.ccw(PT(0,0), p); }

874 PT rotateCCW90(PT p) { return PT(-p.y, p.x); }
222 PT rotateCW90(PT p) { return PT(p.y, -p.x); }
96F PT rotateCCW(PT p, ld t) {
E8C     ld c = cos(t), s = sin(t);
D80     return PT(p.x*c - p.y*s, p.x*s + p.y*c);
93E }

```

3.6 Segment

```

C39 bool onSegment(PT s, PT e, PT p) {
6A6     return p.cross(s, e) == 0 && (s-p) * (e-p) <= 0;
960 }

```

Returns the shortest **distance** between point p and the **segment** s->e.



```

95D ld segmentDist(PT& s, PT& e, PT& p) {
BD2     if (s==e) return (p-s).len();
4B2     ld d = (e-s)*(e-s);
385     ld t = min(d, max<ld>(0, (p-s)*(e-s)));
9E6     return ((p-s)*d - (e-s)*t).len() / d;
A45 }

```

Segment intersection

Unique -> {p}
No inter -> { }
Infinity -> {a, b}, the endpoints of the common segment.
May be rounded if inter isn't integer; Watch out for overflow if long long.

```

3DA int sgn(ll x) { return (x>0) - (x<0); }
FFB vector<PT> segInter(PT a, PT b, PT c, PT d) {
E62     auto oa = c.cross(d, a), ob = c.cross(d, b);
473     auto oc = a.cross(b, c), od = a.cross(b, d);
914     if (sgn(oa)*sgn(ob) < 0 && sgn(oc)*sgn(od) < 0)
E5B         return { (a*ob - b*oa) / (ob-oa) };
529     set<PT> s;
CCB     if (onSegment(c, d, a)) s.insert(a);
0AD     if (onSegment(c, d, b)) s.insert(b);
3D8     if (onSegment(a, b, c)) s.insert(c);
2FA     if (onSegment(a, b, d)) s.insert(d);
C2C     return {begin(s), end(s)};
276 }

```

4 Graphs

4.1 Bridges

```

D44 struct Bridges {
1A8     int n;
789     vector<vector<int>> g;
0CC     vector<pair<int,int>> bridges;
D41
B0A     Bridges(int n) : n(n), g(n) {}
D41
C65     Bridges(const vector<vector<int>>& adj)
5F7         : n(adj.size()), g(adj) { build(); }

```

```

D41 void add_edge(int u, int v) {
58B     g[u].push_back(v);
F2B     g[v].push_back(u);
4D6 }
FFF
D41 void build() {
0A8     vector<int> num(n, -1), low(n);
AC3     int timer = 0;
2DC
D41     function<void(int,int)> dfs = [&](int v, int par
CF3 ) {
006         low[v] = num[v] = timer++;
E32         for (int u : g[v]) {
403             if (num[u] == -1) {
412                 dfs(u, v);
C80                 low[v] = min(low[v], low[u]);
2FE                 if (low[u] > num[v]) bridges.
push_back({v, u});
FB4             } else if (u != par) {
2D3                 low[v] = min(low[v], num[u]);
E66             }
7A3         }
938     };
D41
830     for (int i = 0; i < n; i++)
10B         if (num[i] == -1) dfs(i, -1);
A6F }
000 };

```

4.2 Floyd-Warshall

Floyd-Warshall

Encontra o menor caminho entre todo par de vertices e detecta ciclo negativo
Retorna 1 se ha ciclo negativo
d[i][i] deve ser 0
para i != j, d[i][j] deve ser w se ha uma aresta (i, j)
) de w, INF caso contrario
O(n ao cubo)

```

1A8 int n;
AE5 int d[MAX][MAX];

73C bool floyd_warshall() {
E22     for (int k = 0; k < n; k++)
830         for (int i = 0; i < n; i++)
F90             for (int j = 0; j < n; j++)
0AB                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
D41
830         for (int i = 0; i < n; i++)
753             if (d[i][i] < 0) return 1;
D41
BB3     return 0;
192 }

```

4.3 Toposort

Topological Sort (DFS)

Estados:
0 = nao visitado

```
1 = em processamento
2 = finalizado
```

Complexidade: $O(V + E)$

```
903 vector<vector<int>> adj;
6F9 vector<int> estados;
7B4 stack<int> st;

1EC void toposort(int atual, bool& ciclo) {
749     estados[atual] = 1;
D41
5D6     for (auto vizinho : adj[atual]) {
56C         if (estados[vizinho] == 0) {
EAF             toposort(vizinho, ciclo);
439         } else if (estados[vizinho] == 1) {
D41             // ha ciclo
04B             ciclo = true;
505             return;
A17         }
2DC     }
D41
11F     estados[atual] = 2;
6FB     st.push(atual);
DDD }
```

4.4 Kosaraju

Kosaraju (Grafos fortemente conexos)

g e o grafo (a vai para b)
gi e o grafo reverso (b vai para a)

comp e o componente conexo de cada vertice

graph_condensed() grafo condensado apenas com as
scCs
analisa os graus de entrada e saidas de cadas scCs

$O(m + n)$

```
1A8 int n;
282 vector<int> g[MAXN];
4CA vector<int> gi[MAXN];
465 bool vis[MAXN];
EE6 stack<int> S;
B4B int comp[MAXN];
F97 vector<int> condensed[MAXN];
F9B set<int> scCs;
46F int num_scc = 0;
AC1 map<int, long long> grauEntrada;
F8C map<int, long long> grauSaida;
5A9 long long sink, source;
```

```
1CA void dfs(int k) {
2E7     vis[k] = true;
2BF     for(auto i : g[k]) {
420         if(!vis[i]) {
1E5             dfs(i);
32C         }
407     }
58F     S.push(k);
BFC }
```

```
436 void scc(int k, int c) {
```

```
2E7     vis[k] = true;
52C     comp[k] = c;
947     for(auto i : gi[k]) {
420         if(!vis[i]) {
92F             scc(i, c);
30D         }
36B     }
D24 }
```

```
DB8 void kosaraju() {
E77     for(int i = 0; i < n; i++) vis[i] = false;
158     for(int i = 0; i < n; i++) if(!vis[i]) dfs(i);
D41
E77     for(int i = 0; i < n; i++) vis[i] = false;
980     while (S.size())
F95     {
70B         int u = S.top();
7DE         S.pop();
1D6         if(!vis[u]) {
78F             scc(u, u);
EE7             num_scc++;
71D         }
6D6     }
4C8 }
```

```
05C void graph_condensed() {
D41
830     for (int i = 0; i < n; i++)
F95     {
387         scCs.insert(comp[i]);
223     }
D41
8EB     for (auto i : scCs)
F95     {
D4E         grauEntrada[i] = 0;
D2C         grauSaida[i] = 0;
4E6     }
D41
F92     sink = 0; source = 0;
D41
19E     for (int u = 0; u < n; u++)
F95     {
7B9         for(auto v : g[u]) {
1EC             if(comp[u] != comp[v]) {
55B                 condensed[comp[u]].push_back(comp[v]);
6CF                 grauEntrada[comp[v]]++; grauSaida[comp[u]
]]++;
41B             }
DC4         }
EDD     }
D41
146     for (auto i : grauEntrada)
F95     {
3CE         if(grauEntrada[i.first] == 0) source++;
73A     }
8E3     for (auto i : grauSaida)
F95     {
7C9         if(grauSaida[i.first] == 0) sink++;
417     }
726 }
```

4.5 EulerPath

Euler Path / Euler Cycle

Para declarar: euler<true> E(n) se for direcionado com
N vertices

As funcoes retornam um par
- Booleano indica se ha o path/cycle pedido
- Vetor e formada de {vertice, id aresta para chegar
no vertice}

Se for get_path, na primeira posicao o id vai ser -1
get_path(src) tenta achar um caminho ou ciclo
euleriano, começando no src

#warning chamar para o src certo!

Se achar um ciclo, o primeiro e o ultimo vertice seram
src

Se for um P3, um possivel retorno seria [0, 1, 2, 0]
get_cycle() acha um ciclo euleriano se o grafo for
euleriano

Se for um P3, um possivel retorno seria [0,1,2]
(Vertice inicial nao se repete)

$O(n + m)$

```
63F template<bool directed=false> struct euler {
1A8     int n;
4C0     vector<vector<pair<int, int>>> g;
D63     vector<int> used;
D41
30F     euler(int n_) : n(n_), g(n) {}
50F     void add(int a, int b) {
4CD         int at = used.size();
C51         used.push_back(0);
74E         g[a].emplace_back(b, at);
FAB         if (!directed) g[b].emplace_back(a, at);
411     }
D41
EED     pair<bool, vector<pair<int, int>>> get_path(int src) {
BAF         if (!used.size()) return {true, {}};
B25         vector<int> beg(n, 0);
4EC         for (int& i : used) i = 0;
D41         // {{vertice, anterior}, label}
363         vector<pair<pair<int, int>, int>> ret, st = {{{src,
-1}, -1}};
3C6         while (st.size()) {
8FF             int at = st.back().first.first;
002             int& it = beg[at];
8A1             while (it < g[at].size() and used[g[at][it].second
]) it++;
8E4             if (it == g[at].size()) {
9DD                 if (ret.size() and ret.back().first.second != at
)
B82                     return {false, {}};
420                 ret.push_back(st.back(), st.pop_back());
2C0             } else {
DAA                 st.push_back({{g[at][it].first, at}, g[at][it].
second});
EB8                 used[g[at][it].second] = 1;
D14             }
B3A         }
A19         if (ret.size() != used.size()+1) return {false, {}};
F77         vector<pair<int, int>> ans;
FDF         for (auto i : ret) ans.emplace_back(i.first.first, i
.second);
459         reverse(ans.begin(), ans.end());
997         return {true, ans};
844     }
9B6     pair<bool, vector<pair<int, int>>> get_cycle() {
BAF         if (!used.size()) return {true, {}};
```

```

AD1     int src = 0;
34B     while (!g[src].size()) src++;
687     auto ans = get_path(src);
33C     if (!ans.first or ans.second[0].first != ans.second.
        back().first)
B82         return {false, {}};
350     ans.second[0].second = ans.second.back().second;
8B8     ans.second.pop_back();
BA7     return ans;
48F     }
711 };

```

4.6 BellmanFord

Bellman-Ford
Calcula a menor distancia entre (a) e todos os outros vertices
Detecta ciclo negativo
Retorna 1 se ha ciclo negativo
Nao precisa representar o grafo
So armazenar as arestas

$O(n * m)$

```

14E int n, m;
248 int d[MAX];
45C vector<pair<int, int>> edges;
94A vector<int> weights;

```

```

6BE bool bellman_ford(int a) {
8EC     for (int i = 0; i < n; i++) d[i] = INF;
8A8     d[a] = 0;
D41
4E3     for (int i = 0; i <= n; i++)
891         for (int j = 0; j < m; j++) {
32C             if (d[edges[j].second] > d[edges[j].first] +
                weights[j]) {
705                 if (i == n) return 1;
D41
860                 d[edges[j].second] = d[edges[j].first] + weights
[j];
ACE             }
A99         }
D41
BB3     return 0;
718 }

```

4.7 Dijkstra

Dijkstra - Shortest Paths from Source
Calcula a menor distancia de um vertice s para todos os outros vertices do grafo.
Funciona apenas com pesos ≥ 0 .

Complexity: $O((N + M) \log N)$

grafo[u] = {c, v}
u -> vertice atual
v -> destino

c -> custo da aresta

priority_queue ordena por menor distancia: {distancia, vertice}

```

222 const int INF = 1e18;
B93 using vi = vector<int>;
A06 using pii = pair<int, int>;
62F vector<vector<pii>> grafo;

CDB vi dijkstra(int s, int n) {
1A3     vi dist(n, INF);
B6E     priority_queue<pii, vector<pii>, greater<pii>> fila;
65F     fila.push({0, s});
A93     dist[s] = 0;
D41
74C     while(!fila.empty()) {
FF8         auto [d, u] = fila.top();
8EE         fila.pop();
D41
3E1         if(d > dist[u]) continue;
D41
A95         for(auto [c, v] : grafo[u]) {
511             if( dist[v] > dist[u] + c ) {
085                 dist[v] = dist[u] + c;
CCB                 fila.push({dist[v], v});
B28             }
476         }
372     }
D41
8D7     return dist;
0E7 }

```

4.8 BellmanFordPrintCycle

```

USO:
for (int i = 0; i < m; i++) {
    int u, v, w; cin >> u >> v >> w;
    u--; v--;

    edges.push_back({u, v});
    weights.push_back(w);
}

if (bellman_ford()) {
    cout << "YES\n";
    for (auto x : cycle)
        cout << x + 1 << " ";
}
else {
    cout << "NO\n";
}

```

```

14E int n, m;
EFF ll d[MAX];
922 int parent[MAX];

```

```

45C vector<pair<int, int>> edges;
F24 vector<ll> weights;
411 vector<int> cycle;

```

```

431 bool bellman_ford() {
D41     // dist = 0 em todos => detecta ciclo em qualquer
        componente
603     for (int i = 0; i < n; i++) {

```

```

D0F         d[i] = 0;
CE4         parent[i] = -1;
E3C     }
D6B     int x = -1;
D41
603     for (int i = 0; i < n; i++) {
CD9         x = -1;
D41
891         for (int j = 0; j < m; j++) {
905             int u = edges[j].first;
950             int v = edges[j].second;
02F             ll w = weights[j];
D41
E17             if (d[v] > d[u] + w) {
553                 d[v] = max(-INF, d[u] + w);
ED5                 parent[v] = u;
453                 x = v;
A29             }
E4C         }
CEA     }
D41     // sem ciclo negativo
D41     if (x == -1) return false;
436
D41     // entra dentro do ciclo
D41     for (int i = 0; i < n; i++) x = parent[x];
2D7     int cur = x;
D41
016     do {
A10         cycle.push_back(cur);
D77         cur = parent[cur];
A92     }
20C     while (cur != x);
CEF     cycle.push_back(x);
563     reverse(cycle.begin(), cycle.end());
D41
8A6     return true;
157 }

```

4.9 Prim

Prim - Minimum Spanning Tree
Algoritmo para encontrar a Arvore Geradora Minima (MST)

Estrategia:
-> Expande a arvore sempre pegando a menor aresta disponivel

Complexidade: $O(E \log V)$

E = numero de arestas; V = numero de vertices

```

051 vector<int> vis;
63C vector<vector<pair<int, int>>> adj;

```

```

518 ll prim(int atual) {
10B     priority_queue<pair<int, int>, vector<pair<int, int>
        >>, greater<pair<int, int>>> mh;
E39     mh.push({0, atual});
D41
FEC     ll custo = 0;
D41
5ED     while (!mh.empty()) {
4C4         auto [c, u] = mh.top(); mh.pop();

```

```

D41         if (!vis[u]) {
D46             vis[u] = true;
B9C             custo = custo + c;
D41
D5D             for (auto [v, peso] : adj[u]) {
D90                 if (!vis[v]) {
D90                     mh.push({peso, v});
5CE                 }
O20             }
588         }
116     }
A25 }
D41
F54     return custo;
408 }

```

4.10 BFS

```

OE7 int grid[1000][1000];

D41 // R D U L
FDC int dx[] = {0, 1, -1, 0};
6BE int dy[] = {1, 0, 0, -1};

D41 // R D U L DR DL UR UL
59A int dx[] = {0, 1, -1, 0, 1, 1, -1, -1};
6E7 int dy[] = {1, 0, 0, -1, 1, -1, 1, -1};

93A bool visitados[1000][1000];

E3F void BFS(int x, int y) {
155     queue<pair<int,int>> q;
987     q.push({x,y});
D41
09A     visitados[x][y] = true;
D41
402     while(q.size()){
3DB         auto [x1, y1] = q.front();
833         q.pop();
D41
1CD         for(int i = 0; i < 4; i++){
AE4             int ax = x1 + dx[i];
C83             int ay = y1 + dy[i];
D41
906             if(!visitados[ax][ay]){
237                 visitados[ax][ay] = true;
41A                 q.push({ax, ay});
A15             }
5B3         }
BA8     }
C6B }

```

4.11 DFS

```

OE7 int grid[1000][1000];

D41 // R D U L
FDC int dx[] = {0, 1, -1, 0};
6BE int dy[] = {1, 0, 0, -1};

D41 // R D U L DR DL UR UL
59A int dx[] = {0, 1, -1, 0, 1, 1, -1, -1};
6E7 int dy[] = {1, 0, 0, -1, 1, -1, 1, -1};

```

```

93A bool visitados[1000][1000];

COD void dfs(int x, int y){
09A     visitados[x][y] = true;
D41
1CD     for(int i = 0; i < 4; i++){
F82         int ax = x + dx[i];
EE1         int ay = y + dy[i];
D41
2CC         if(!visitados[ax][ay]) dfs(ax,ay);
0AE     }
7E8 }

```

4.12 DSU

Disjoint Set Union - Union Find
Find: $O(a(n)) \rightarrow$ Inverse Ackermann function
Join: $O(a(n)) \rightarrow a(1e6) \leq 5$

```

D56 struct DSU {
B15     vector<int> pai, sz;
D5C     DSU(int n) : pai(n+1), sz(n+1, 1) {
51E         for(int i=0; i<=n; i++) pai[i] = i;
A82     }
D41
F6C     int find(int u){ return pai[u] == u ? u : pai[u] =
find(pai[u]); }
D41
AF9     void join(int u, int v){
B80         u = find(u), v = find(v);
D41
360         if(u == v) return;
844         if(sz[v] > sz[u]) swap(u, v);
D41
555         pai[v] = u;
CC3         sz[u] += sz[v];
32C     }
175 };

```

4.13 MinCostMaxFlow

MCMF find the maximum possible flow from a source to a sink while ensuring the total cost of flow is minimized
Cost per unit of flow
Capacity
 $O(\text{Total Flow} + (\text{Edges} + \text{Nodes}) \log \text{Nodes})$

```

7C9 struct Aresta {
F0B     int u, v; ll cap, cost;
FF2     Aresta(int u, int v, ll cap, ll cost) : u(u), v(v),
cap(cap), cost(cost) {}
1D9 };

6F3 struct MCMF {
878     const ll INF = numeric_limits<ll>::max();
6B0     int n, source, sink;
903     vector<vector<int>> adj;
4DF     vector<Aresta> edges;
39D     vector<ll> dist, pot;
E3B     vector<int> from;
D41

```

```

D98     MCMF(int n, int source, int sink) : n(n), source(
source), sink(sink) { adj.resize(n); pot.resize(n); }
D41
3A2     void addAresta(int u, int v, ll cap, ll cost){
471         adj[u].push_back(edges.size());
986         edges.emplace_back(u, v, cap, cost);
D41
282         adj[v].push_back(edges.size());
29F         edges.emplace_back(v, u, 0, -cost);
D21     }
D41
26A     queue<int> q;
B57     vector<bool> vis;
791     bool SPFA(){
EF2         dist.assign(n, INF);
0B5         from.assign(n, -1);
543         vis.assign(n, false);
D41
7CD         q.push(source);
506         dist[source] = 0;
D41
14D         while(!q.empty()){
E4A             int u = q.front();
833             q.pop();
D41
776             vis[u] = false;
D41
E20             for(auto i : adj[u]){
F42                 if(edges[i].cap == 0) continue;
628                 int v = edges[i].v;
99A                 ll cost = edges[i].cost;
D41
148                 if(dist[v] > dist[u] + cost + pot[u] - pot[v]){
DEC                     dist[v] = dist[u] + cost + pot[u] - pot[v];
203                     from[v] = i;
A1A                     if(!vis[v]) q.push(v), vis[v] = true;
888                 }
652             }
344         }
D41
19E         for(int u=0; u<n; u++){
067             if(dist[u] < INF)
AB7                 pot[u] += dist[u];
D41
9DE         return dist[sink] < INF;
D50     }
D41
B84     pair<ll, ll> augment(){
B3F         ll flow = edges[from[sink]].cap, cost = 0;
940         for(int v=sink; v != source; v = edges[from[v]].u)
73D             flow = min(flow, edges[from[v]].cap),
871             cost += edges[from[v]].cost;
D41
940         for(int v=sink; v != source; v = edges[from[v]].u)
86A             edges[from[v]].cap -= flow,
674             edges[from[v]^1].cap += flow;
D41
884         return {flow, cost};
668     }
D41
164     bool inCut(int u){ return dist[u] < INF; }
D41
6DC     pair<ll, ll> maxFlow(){
D7D         ll flow = 0, cost = 0;
D41
4EB         while( SPFA() ){
274             auto [f, c] = augment();
C87             flow += f;
BFC             cost += f*c;
35C         }

```

```

884     return {flow, cost};
D37 }
22E };

```

4.14 EulerTour

Euler Tour
Lineariza um grafo
Verificar a ancestralidade de v se $in[u]$
 $\leq in[v] \leq out[u]$

```

7F3 #define MAXN 100

271 bool visitados[MAXN];

171 void EulerTour(int u, vector<vector<int>>& adj, vector<
    int>& euler){
83D     euler.push_back(u);
2D1     visitados[u] = true;
D41
D1C     for(auto v : adj[u]){
C1B         if(!visitados[v]){
544             EulerTour(v, adj, euler);
83D             euler.push_back(u);
68E         }
02D     }
CC9 }

```

4.15 MinCostMaxFlowSimple

MCMF Simplificado
Achar o MinCut e rodar a bfs mais uma vez pra checar
quem nao e mais reachable (Tem capacidade menor que
0)
MinCut == Flow == Max_flow

```

0C1 #define vi vector<int>
9F0 #define INF 100000000

698 vector<vi> adj;
7D4 vector<vi> capacidade; // capacidade[u][v] = cap. da
    aresta u -> v
3FF vi pai;

B57 vector<bool> vis;
1B1 bool bfs(int source, int sink){
D41
FCC     fill(pai.begin(), pai.end(), -1);
B00     fill(vis.begin(), vis.end(), false);
26A     queue<int> q;
7CD     q.push(source);
D41
22C     vis[source] = true;
14D     while(!q.empty()){
E4A         int u = q.front();
833         q.pop();
D41
372         for(int v : adj[u]){
B10             int c = capacidade[u][v];
EA5             if(c <= 0 || pai[v] != -1 || vis[v]) continue;

```

```

CCA         vis[v] = 1;
555         pai[v] = u;
2A1         q.push(v);
C10     }
7AE     }
0E9     return pai[sink] != -1;
9E6 }

FDF ll maxflow(int source, int sink){
7A2     ll flow = 0;
D41
5D6     while(bfs(source, sink)){
B93         int u = sink;
CBD         int curr_flow = INF;
D41
96C         while(u != source){
2F6             curr_flow = min(curr_flow, capacidade[pai[u]][u]);
3CF             u = pai[u];
AA3         }
D41
26E         u = sink;
D41
96C         while(u != source){
D41             // augment
716             capacidade[pai[u]][u] -= curr_flow;
0DB             capacidade[u][pai[u]] += curr_flow;
3CF             u = pai[u];
4C8         }
D41
C76         flow += curr_flow;
CD5     }
D41
99D     return flow;
F73 }

```

4.16 LCA

```

D41 /* LCA (Lowest Common Ancestor)
685 * Encontra o ancestral comum mais proximo de dois
    vertices em uma arvore.
025 * Complexidade: O(N log N) no build, O(1) por query.
606 * Memoria: O(N log N)
C7E * Requisitos:
4F4 * - Grafo deve ser uma arvore (conexo e aciclico).
544 * - Dependencia: sparse-table/sparse-table.hpp
15D * Metodos:
4F8 * - lca(u, v): Retorna o LCA de u e v.
594 * - dist(u, v): Retorna a distancia entre u e v.
992 * - is_ancestor(u, v): Verifica se u e ancestral de v.
551 * - parent(u): Retorna o pai de u.
C4C */

```

```

8A0 template<typename NODE>
7E9 struct SparseTable {
5F0     int N, K;
6F5     vector<vector<NODE>> st;
4A7     vector<int> lg;
D41
671     SparseTable() : N(0), K(0) {}
D41
2EB     SparseTable(const vector<NODE>& v) : N(v.size()) {
3AA         if (N == 0) return;
906         K = __lg(N);
3A9         st.assign(K + 1, vector<NODE>(N));
641         lg.assign(N + 1, 0);

```

```

BB5     for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] +
1;
D41
FA8     for (int i = 0; i < N; i++) st[0][i] = v[i];
D41
2CE     for (int j = 1; j <= K; j++)
285         for (int i = 0; i + (1 << j) <= N; i++)
4E1             st[j][i] = NODE::merge(st[j - 1][i],
A64                                     st[j - 1][i + (1
<< (j - 1))]);
6D4     }
D41
762     NODE query(int l, int r) {
2C3         int j = lg[r - l + 1];
8F2         return NODE::merge(st[j][l], st[j][r - (1 << j)
+ 1]);
843     }
51F };

542 struct LCANode {
43D     int dep, id;
89D     LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos)
{}
D41
5CF     static inline LCANode merge(const LCANode& l, const
LCANode& r) {
DC4         return l.dep < r.dep ? l : r;
505     }
DAD };

33E struct LCA {
060     int N;
41B     vector<int> first, tour, d, p;
73B     SparseTable<LCANode> st;
D41
D41     //(1-based indexing)
DCD     LCA(int n, int root, const vector<vector<int>>& adj)
4EC         : N(n), first(n + 1), d(n + 1), p(n + 1, -1), st
(vector<LCANode>()) {
D41
4B8         vector<LCANode> euler_nodes;
657         tour.reserve(2 * n + 5);
9DF         euler_nodes.reserve(2 * n + 5);
D41
auto dfs = [&](auto self, int u, int parent_id,
int dep) -> void {
73F         p[u] = parent_id;
701         d[u] = dep;
9D0         first[u] = euler_nodes.size();
D41
FD8         tour.push_back(u);
B74         euler_nodes.push_back(LCANode(dep, u));
D41
98F         for (int next_node : adj[u]) {
6B9             if (next_node == parent_id) continue;
0DF             self(self, next_node, u, dep + 1);
D41
FD8         tour.push_back(u);
B74         euler_nodes.push_back(LCANode(dep, u));
F3F     }
26D     };
D41
7D9     dfs(dfs, root, -1, 0);
D41
A4C     st = SparseTable<LCANode>(euler_nodes);
5B1 }
D41
int parent(int u) {
F60     return p[u];
47F }

```

```

D41      int lca(int u, int v) {
B63          int l = first[u], r = first[v];
A13          if (l > r) swap(l, r);
DBB          return st.query(l, r).id;
9CC      }
D41
C11      int dist(int u, int v) {
O5C          return d[u] + d[v] - 2 * d[lca(u, v)];
465      }
D41
F31      bool is_ancestor(int u, int v) {
645          return lca(u, v) == u;
C22      }
OC7  };

```

4.17 Kruskal

Kruskal - Minimum Spanning Tree
 Algoritmo para encontrar a Arvore Geradora Minima (MST)
 -> Complexity: $O(E \log E)$
 E : Numero de Arestas

```

D41 /*Create a DSU*/
229 const int MAXN = 1e6 + 5;
264 struct Aresta{ int u, v, c; };
1DB bool compAresta(Aresta a, Aresta b){ return a.c < b.c; }

B83 vector<Aresta> arestas;

D1E int kruskal(int n){
76A     sort(begin(arestas), end(arestas), compAresta);
370     DSU dsu(n);
D41
3BD     int resp = 0;
O31     for(auto a : arestas) {
F3C         if (find(a.u) != find(a.v)) {
801             resp += a.c;
24B             join(a.u, a.v);
A75         }
248     }
68A     return resp;
F4C }

```

5 dp

5.1 editDistance

```

3C1 long long editDistance(const string& s1, const string&
EFB      s2,                                long long a, long long b,
      long long c, long long d) {
13A     int n = s1.length();
8E1     int m = s2.length();
D41
E06     vector<vector<long long>> memo(n + 1, vector<long
long>(m + 1, 0));
D41
CCD     for (int i = 0; i <= n; i++) {
241         for (int j = 0; j <= m; j++) {
D41

```

```

D41      //caso base: se a segunda string vazia,
      apenas delecoes
FDB      if (j == 0) {
D9A          memo[i][j] = i * b;
B00      }
D41      //caso base: se a primeira string vazia,
      apenas insercoes
DF6      else if (i == 0) {
D60          memo[i][j] = j * a;
ABF      }
D41      //caracteres iguais, custo 0
5FC      else if (s1[i - 1] == s2[j - 1]) {
7AE          memo[i][j] = memo[i - 1][j - 1];
D3B      }
D41      //caracteres diferentes, calcula o menor
      custo operacional
4E6      else {
129          long long op_insercao = memo[i][j - 1] +
a;
21E          long long op_delecao = memo[i - 1][j] +
b;
7CE          long long op_substituicao = memo[i - 1][
j - 1] + c;
D41
B3E          memo[i][j] = min({op_insercao,
op_delecao, op_substituicao});
D41
D41      //verifica se a Transposicao e possivel
e se ela e mais barata
AA4      if (i >= 2 && j >= 2 && s1[i - 1] == s2[
j - 2] && s1[i - 2] == s2[j - 1]) {
954          long long op_transposicao = memo[i -
2][j - 2] + d;
469          memo[i][j] = min(memo[i][j],
op_transposicao);
626      }
119      }
34A      }
947      }
D41
372      return memo[n][m];
E53 }

```

5.2 LIS

LIS - Longest Increasing Subsequence

Complexity: $O(N \log N)$
 * For INCREASING sequence, use lower_bound()
 * For NON DECREASING sequence, use upper_bound()

```

7A6 int LIS(vector<int>& nums){
OFF     vector<int> lis;
D41
549     for(auto x : nums)
F95     {
3D0         auto it = lower_bound(lis.begin(), lis.end(), x);
D41
CDF         if(it == lis.end()) lis.push_back(x);
77C         else *it = x;
738     }
D41
737     return (int) lis.size();
F27 }

```

5.3 subsetSum

Subset sum
 Retorna max(x <= t tal que existe subset de w que soma x)

Complexidade
 $O(n * \max(w))$
 $O(\max(w))$ de memoria

```

EFD int subset_sum(vector<int> w, int t) {
BB5     int pref = 0, k = 0;
417     while (k < w.size() and pref + w[k] <= t) pref += w[k]
++;
1E7     if (k == w.size()) return pref;
444     int W = *max_element(w.begin(), w.end());
44D     vector<int> last, dp(2*W, -1);
D7B     dp[W - (t-pref)] = k;
54D     for (int i = k; i < w.size(); i++) {
288         last = dp;
15F         for (int x = 0; x < W; x++) dp[x+w[i]] = max(dp[x+w[
i]], last[x]);
17B         for (int x = 2*W - 1; x > W; x--)
303             for (int j = max(0, last[x]); j < dp[x]; j++)
595                 dp[x-w[j]] = max(dp[x-w[j]], j);
867     }
2FB     int ans = t;
1C1     while (dp[W - (t-ans)] < 0) ans--;
BA7     return ans;
D88 }

```

5.4 LCS

LCS - Longest Common Subsequence

Complexity: $O(N^2)$

* Recursive: memset(memo, -1, sizeof memo); LCS(0, 0);
 * Iterative: LCS_It();

* RecoverLCS $O(N)$
 Recover just one of all the possible LCS

```

A2C const int MAXN = 5*1e3 + 5;
DD0 int memo[MAXN][MAXN];

AC1 string s, t;

478 inline int LCS(int i, int j){
BF8     if(i == s.size() || j == t.size()) return 0;
B5D     if(memo[i][j] != -1) return memo[i][j];
D41
O52     if(s[i] == t[j]) return memo[i][j] = 1 + LCS(i+1, j+1)
;
D41
A17     return memo[i][j] = max(LCS(i+1, j), LCS(i, j+1));
F66 }

DBD string RecoverLCS(int i, int j){
F34     if(i == s.size() || j == t.size()) return "";
D41
134     if(s[i] == t[j]) return s[i] + RecoverLCS(i+1, j+1);

```

```

D41
495     if(memo[i+1][j] > memo[i][j+1]) return RecoverLCS(i+1,
j);
D41
DCC     return RecoverLCS(i, j+1);
5E7 }

```

5.5 SumOverSubsetDP

SOS DP [nohash]

Soma de sub-conjunto e de super-conjunto
 $O(n \cdot 2^n)$

```

2F3 vector<ll> sos_dp_sub(vector<ll> f) {
6C0     int N = __builtin_ctz(f.size());
E59     assert((1<<N) == f.size());
D41
5A5     for (int i = 0; i < N; i++) for (int mask = 0; mask <
(1<<N); mask++)
796         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
ABE     return f;
818 }

```

```

171 vector<ll> sos_dp_sup(vector<ll> f) {
6C0     int N = __builtin_ctz(f.size());
E59     assert((1<<N) == f.size());
D41
5A5     for (int i = 0; i < N; i++) for (int mask = 0; mask <
(1<<N); mask++)
A3C         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
ABE     return f;
5AE }

```

5.6 knapsack

Resolve mochila, recuperando a resposta
 DP usando os itens [1, r], com capacidade = cap
 v[max] e w[MAX] valor e peso

Complexidade:
 $\rightarrow O(n \cdot \text{cap}), O(n + \text{cap})$

```

857 #define MAX (long long) 1e4
69E #define MAX_CAP (long long) 1e4
66F #define INF INT_MAX
ADD int v[MAX], w[MAX];
582 int dp[2][MAX_CAP];
OD6 void get_dp(int x, int l, int r, int cap) {
F8F     memset(dp[x], 0, (cap+1)*sizeof(dp[x][0]));
574     for (int i = l; i <= r; i++) for (int j = cap; j >= 0;
j--)
3A9         if (j - w[i] >= 0) dp[x][j] = max(dp[x][j], v[i] +
dp[x][j - w[i]]);
B95 }

```

```

5AB void solve(vector<int>& ans, int l, int r, int cap) {
893     if (l == r) {
9FF         if (w[l] <= cap) ans.push_back(l);
505         return;
13A     }
EE4     int m = (l+r)/2;
283     get_dp(0, l, m, cap), get_dp(1, m+1, r, cap);
056     int left_cap = -1, opt = -INF;
C94     for (int j = 0; j <= cap; j++)
2F2         if (int at = dp[0][j] + dp[1][cap - j]; at > opt)
91D             opt = at, left_cap = j;
DA3     solve(ans, l, m, left_cap), solve(ans, m+1, r, cap -
left_cap);
D75 }
OD7 vector<int> knapsack(int n, int cap) {
DAB     vector<int> ans;
1E0     solve(ans, 0, n-1, cap);
BA7     return ans;
E4D }

```

6.2 hash

6 Strings

6.1 Manacher

Manacher Algorithm
 Find every palindrome in string
 Complexidade: $O(N)$

```

DC6 vector<int> manacher(string &st){
E13     string s = "$_";
821     for(char c : st){ s += c; s += "_"; }
095     s += "##";
D41
995     int n = s.size()-2;
D41
BD7     vector<int> p(n+2, 0);
7CD     int l=1, r=1;
D41
557     for(int i=1, j; i<=n; i++)
F95     {
DAF         p[i] = max(0, min(r-i, p[l+r-i])); //atualizo o
valor atual para o valor do palindromo espelho na
string ou para o total que esta contido
D41
A5F         while( s[i-p[i]] == s[i+p[i]] ) p[i]++;
D41
39C         if( i+p[i] > r ) l = i-p[i], r = i+p[i];
A83     }
D41
6AE     for(auto &x : p) x--; //o valor de p[i] e igual ao
tamanho do palindromo + 1
D41
74E     return p;
907 }

```

```
String Hash
precalc() -> O(N)
StringHash() -> O(|S|)
gethash() -> O(1)
```

StringHash hash(s); -> Cria uma struct de StringHash para a string s
hash.gethash(l, r); -> Retorna o hash do intervalo l R da string (0-Indexado)

IMPORTANTE! Chamar precalc() no inicio do codigo

const ll MOD = 131'807'699; -> Big Prime Number
const ll base = 127; -> Random number larger than the Alphabet

```
229 const int MAXN = 1e6 + 5;
```

```
622 const ll MOD = 1e9 + 7; //WA? Muda o MOD e a base
AA0 const ll base = 153;
```

```
7E6 ll expb[MAXN];
```

```
FE8 void precalc(){
5A7 expb[0] = 1;
7E4 for(int i=1; i<MAXN; i++)
3C6 expb[i] = (expb[i-1]*base)%MOD;
7FF }
```

```
3CE struct StringHash{
894 vector<ll> hsh;
D41
B4B StringHash(string &s){
4E1 hsh.assign(s.size()+1, 0);
724 for(int i=0; i<s.size(); i++)
24D hsh[i+1] = (hsh[i] * base % MOD + s[i]) % MOD;
9E9 }
D41
A5B ll gethash(int l, int r){
794 return (MOD + hsh[r+1] - hsh[l]*expb[r-l+1] % MOD )
% MOD;
C17 }
FFA };
```

6.3 trie

Trie - Arvore de Prefixos
insert(P) - O(|P|)
count(P) - O(|P|)
MAXS - Soma do tamanho de todas as Strings
sigma - Tamanho do alfabeto

```
AAF const int MAXS = 1e5 + 10;
70C const int sigma = 26;
```

```
F6C int trie[MAXS][sigma], terminal[MAXS], z = 1;
```

```
33B void insert(string &p){
B3D int cur = 0;
D41
E2E for(int i=0; i<p.size(); i++){
1BF int id = p[i] - 'a';
D41
BCF if(terminal[cur][id] == -1){
616 memset(terminal[cur][id], -1, sizeof terminal[cur][id]);
```

```
869 trie[cur][id] = z++;
CAE }
D41
3AD cur = trie[cur][id];
A9E }
D41
B07 terminal[cur]++;
C89 }

684 int count(string &p){
B3D int cur = 0;
D41
E2E for(int i=0; i<p.size(); i++){
94B int id = (p[i] - 'a');
D41
C39 if(terminal[cur][id] == -1) return 0;
D41
3AD cur = trie[cur][id];
ADB }
89E return terminal[cur];
D3C }
```

```
CA2 void init(){
E6F memset(terminal, -1, sizeof terminal);
34E z = 1;
A11 }
```

6.4 KMP

KMP - Find all occurrences of a pattern string inside a text string

matching(s, t) retorna os indices das ocorrencias de s em t
autKMP constroi o automato do KMP

Complexidades:
pi - O(n)
match - O(n + m)
construir o automato - O(|sigma|*n)
n = |padrao| e m = |texto|

```
EA8 template<typename T> vector<int> pi(T s) {
019 vector<int> p(s.size());
725 for (int i = 1, j = 0; i < s.size(); i++) {
A51 while (j and s[j] != s[i]) j = p[j-1];
973 if (s[j] == s[i]) j++;
F8C p[i] = j;
E0A }
74E return p;
F50 }

C10 template<typename T> vector<int> matching(T& s, T& t) {
658 vector<int> p = pi(s), match;
A1B for (int i = 0, j = 0; i < t.size(); i++) {
6BE while (j and s[j] != t[i]) j = p[j-1];
C4D if (s[j] == t[i]) j++;
310 if (j == s.size()) match.push_back(i-j+1), j = p[j-1];
028 }
ED8 return match;
C82 }
```

```
A2D struct KMPaut : vector<vector<int>> {
47C KMPaut() {}
```

```
6C7 KMPaut (string& s) : vector<vector<int>>(26, vector<
int>(s.size()+1)) {
503 vector<int> p = pi(s);
04B auto& aut = *this;
4FA aut[s[0]-'a'][0] = 1;
19A for (char c = 0; c < 26; c++)
5D3 for (int i = 1; i <= s.size(); i++)
42B aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i]-1];
4BB }
79B };
```

7 Math

7.1 totient

Totiente de Euler - Conta quantos numeros de 1 ate n sao coprimos de n

Complexidade:
O(sqrt(n))

```
D41 // Totiente
D41 //
D41 // O(sqrt(n))
```

```
A7E int tot(int n) {
0F6 int ret = n;
D41
505 for (int i = 2; i*i <= n; i++) if (n % i == 0) {
B0C while (n % i == 0) n /= i;
125 ret -= ret / i;
34A }
AF4 if (n > 1) ret -= ret / n;
D41
EDF return ret;
FAE }
```

7.2 sieve

Sieve of Eratosthenes - Encontra o maior divisor primo

Fact -> Fatora um numero <= limite, sai ordenada
Crivo calcula a lista de primos

Crivo_mobius
- 1 se n=1
- 0 se n tem algum fator primo ao quadrado
- (-1)^k se n e produto de k primos distintos

A funcao fact adiciona o numero 1 se vc tentar fatorar o 1.

Complexidade:
crivo - O(n log(logN))
fact - O(log(n))

```
276 #define MAX 1000
F12 int divi[MAX];
FD3 vector<int> primes;
```

```

FB9 void crivo(int lim) {
D5A     divi[1] = 1;
F70     for (int i = 2; i <= lim; i++) {
3EB         if (divi[i] == 0) divi[i] = i, primes.push_back(i);
3BA         for (int j : primes) {
522             if (j > divi[i] or i*j > lim) break;
00B             divi[i*j] = j;
491         }
85A     }
519 }

4C0 void crivo_mobius(ll lim) {
A7D     mobius[1] = 1;
F70     for (int i = 2; i <= lim; i++) {
4DB         if (!divi[i]) {
DFA             divi[i] = i;
E74             primes.push_back(i);
F69             mobius[i] = -1;
2AC         }
3B9         for (int p : primes) {
5EB             if (p > divi[i] || 1LL * i * p > lim) break;
DDC             divi[i*p] = p;
569             if (i % p == 0) {
823                 mobius[i*p] = 0;
C2B                 break;
5D6             } else {
2F7                 mobius[i*p] = -mobius[i];
482             }
31F         }
A50     }
BA0 }

470 void fact(vector<int>& v, int n) {
AC8     if (n != divi[n]) fact(v, n/divi[n]);
AB4     v.push_back(divi[n]);
1DB }

```

7.3 Kadane

Algoritmo de Kadane
Consegue o max subarray sum
 $O(N)$

```

F77 ll kadane(vector<ll> &arr) {
25A     ll ans = arr[0];
9A2     ll maxEnding = arr[0];
D41
B37     for (int i = 1; i < arr.size(); i++) {
9BC         maxEnding = max(maxEnding + arr[i], arr[i]);
D41
0C0         ans = max(ans, maxEnding);
A3B     }
D41
D72     return ans;
6E6 }

```

7.4 trianguloPascal

```
3E4 const int MOD = 1e9 + 7;
```

```
D41 //O(N ao quadrado)
```

```

D41 //Pre-computa os coeficientes binomiais (combinacoes)
      usando o Triangulo de Pascal.
D41 //De quantas formas posso escolher k elementos distintos
      a partir de um conjunto de n elementos?
D41 //limite O valor maximo de 'n' que voce deseja calcular
      (geralmente determinado por 'maxi').

614 vector<vector<int>> precompute_combinacoes(int limite) {
D41
DAC     vector<vector<int>> tabela_pascal(limite + 1, vector
      <int>(limite + 1, 0));
D41
FD5     for (int i = 0; i <= limite; i++) {
A00         tabela_pascal[i][0] = 1;
A63     }
D41
DDB     for (int i = 1; i <= limite; i++) {
806         for (int j = 1; j <= i; j++) {
2B4             tabela_pascal[i][j] = (tabela_pascal[i - 1][
      j - 1] + tabela_pascal[i - 1][j]) % MOD;
F52         }
56C     }
D41
67C     return tabela_pascal;
7B7 }

```

7.5 divisors

get_divisors(n) -- $O(\sqrt{N})$

```

EC9 vector<int> get_divisors(int n) {
921     vector<int> divisors;
D41
671     for(int i = 1; i*i <= n; i++){
C06         if(n % i == 0){
025             divisors.push_back(i);
529             if(i != n/i) divisors.push_back(n/i);
B4D         }
E12     }
E83     return divisors;
499 }

```

7.6 ModComb

Combinacao modular
Inverso modular
Exponenciacao rapida ($O(\log P)$) - p: potencia
 $O(N)$ fatorial

```
738 #define MOD 9987123
```

```
60F vector<ll> fact(1e6, -1);
```

```

165 void pre() {
0C0     fact[0] = 1;
399     for(ll i = 1; i < fact.size(); i++) {
DA2         fact[i] = (fact[i-1] * i) % MOD;
901     }
3F9 }

```

```

113 ll fexp(ll a, ll b){
D54     ll ans = 1;
D41

```

```

1B4     while(b){
98C         if(b & 1) ans = (ans * a) % MOD;
49B         a = (a*a) % MOD;
1B4         b >>= 1;
771     }
D41
BA7     return ans;
D24 }

2E9 ll inv(ll a, ll p){
284     return fexp(a, p - 2);
B8F }

DAB ll comb(ll n, ll k, ll p){
44E     return ((fact[n] * inv(fact[k], p)) % p) * inv(fact[
      n-k], p) % p;
AA0 }

```

8 Theorems

8.1 Propriedades Matemáticas

- **Lagrange:** Todo número inteiro pode ser representado como soma de 4 quadrados.
- **Zeckendorf:** Todo número pode ser representado como soma de números de Fibonacci diferentes e não consecutivos.
- **Tripla de Pitágoras (Euclides):** Toda tripla pitagórica primitiva pode ser gerada por $(n^2 - m^2, 2nm, n^2 + m^2)$ onde n e m são coprimos e um deles é par.
- **Progressão Geométrica:** $S_n = a_1 \cdot \frac{q^n - 1}{q - 1}$
- **Soma de quadrados e cubos:** $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$
- **Chicken McNugget:** Para dois coprimos x e y , o número de inteiros que não podem ser expressos como $ax + by$ é $(x-1)(y-1)/2$. O maior inteiro não representável é $xy - x - y$.
- **Primos até 200:** 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199
- **Fibonacci [0, 20]:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765.
- **Fatorial [0, 15]:** 1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800, 39916800, 479001600, 6227020800, 87178291200, 1307674368000.

Potencias

2^n [1, 16]: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536

3^n [1, 13]: 3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049, 177147, 531441, 1594323

5^n [1, 12]: 5, 25, 125, 625, 3125, 15625, 78125, 390625, 1953125, 9765625, 48828125, 244140625

7^n [1, 9]: 7, 49, 343, 2401, 16807, 117649, 823543, 5764801, 40353607, 282475249

Totiente de Euler: $\varphi(n)$ conta quantos números entre 1 e n são coprimos com n .

Seq.[1, 32]: 1, 1, 2, 2, 4, 2, 6, 4, 10, 4, 12, 6, 8, 8, 16, 6, 18, 8, 12, 10, 22, 8, 20, 12, 18, 12, 28, 8, 30

Primos

- Goldbach:** Todo número par $n > 2$ pode ser representado como $n = a + b$, onde a e b são primos.
- Primos Gêmeos:** Existem infinitos pares de primos p , $p + 2$.
- Legendre:** Sempre existe um primo entre n^2 e $(n + 1)^2$.
- Wilson:** n é primo se e somente se $(n - 1)! \mod n = n - 1$.

Combinatória

Propriedades de Coeficientes Binomiais:

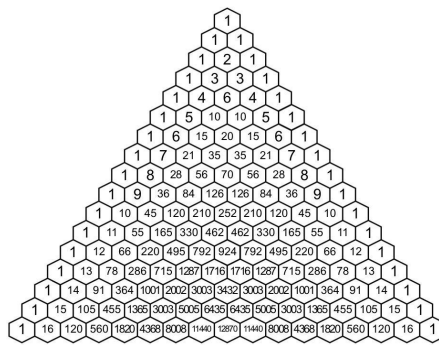
$$\binom{n}{k} = \binom{n}{n-k} = \frac{n}{k} \binom{n-1}{k-1},$$

$$\sum_{i=0}^k \binom{m}{i} \binom{n}{k-i} = \binom{m+n}{k}, \sum_{i=0}^m \binom{n}{i} (-1)^i = \binom{n-1}{m} (-1)^m$$

$$\sum_{i=0}^n \binom{n}{i} = 2^n, \quad \sum_{i=0}^n \binom{n}{i} i = n \cdot 2^{n-1},$$

$$\sum_{i=0}^n \binom{n-i}{i} = F_{n+1}, \quad \sum_{i=0}^m \binom{n+i}{i} = \binom{n+m+1}{m},$$

Triângulo de Pascal



- Números de Catalan:** expressões de parênteses bem formadas, #binary trees com $n+1$ nodes, etc. $C_0 = 1$, e:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

Seq.[0, 20]: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420.

- Burnside Lemma:** Número de colares diferentes (sem contar rotações), com m cores e comprimento n :

$$\frac{1}{n} \left(m^n + \sum_{i=1}^{n-1} m^{\gcd(i,n)} \right)$$

- Inversão de Möbius:** Útil para inclusão e exclusão nos primos. Seq.[1, 30]: 1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0, 1, 1, 0, -1, 1, 0, 0, 1, -1, 1, 0, -1, -1, 0, 1, 0, 0, 1. $\sum_{d|n} \mu(d) = 0$ if $n > 1$ else 1

- Lindström-Gessel-Viennot:** A quantidade de caminhos disjuntos em um grid pode ser computada como o determinante da matriz do número de caminhos.

Números de Stirling:

- First Type:** $|s(n, m)|$ é definido como a quantidade de permutações de n elementos que contêm exatamente m ciclos de permutação. Tem aplicações em inclusão e exclusão.

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} = (-1)^{n-k} s(n, k)$$

$$\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)! \quad , \quad \begin{bmatrix} n \\ n \end{bmatrix} = 1 \quad , \quad \begin{bmatrix} n+1 \\ m+1 \end{bmatrix} = \sum_i \begin{bmatrix} n \\ i \end{bmatrix} \begin{bmatrix} i \\ m \end{bmatrix}$$

$n k$	0	1	2	3	4	5	6
0	1						
1		1					
2		-1	1				
3		2	-3	1			
4		-6	11	-6	1		
5		24	-50	35	-10	1	
6		-120	274	-225	85	-15	1

- Second Type:** O Stirling set $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ conta maneiras de particionar um conjunto de n elementos em k sub-conjuntos não vazios. Equivalente a Rhyming Schemes de n posições e k símbolos. (exemplo $n=3$: $k=1$ {aaa}; $k=2$ {aab, aba, abb}; $k=3$ {abc})

$$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$$

$$\left\{ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n \\ n \end{smallmatrix} \right\} = 1 \quad , \quad \left\{ \begin{smallmatrix} n+1 \\ k+1 \end{smallmatrix} \right\} = \sum_i \left\{ \begin{smallmatrix} i \\ k \end{smallmatrix} \right\} \binom{n}{i}$$

$n k$	0	1	2	3	4	5	6
0	1						
1		1					
2		1	1				
3		1	3	1			
4		1	7	6	1		
5		1	15	25	10	1	
6		1	31	90	65	15	1

- Números de Bell B_n :** numero de partições de um conjunto de n elementos. $B_n = \sum_i \left\{ \begin{smallmatrix} n \\ i \end{smallmatrix} \right\} = \sum_i \binom{n-1}{i} B_i$. [0, 12]: 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, 4213597.

Eulerian numbers (first order):

O Eulerian number $\langle n \rangle_k$ ou $A(n, k)$ conta o número de permutações de tamanho n com exatamente k ascents ($p_i < p_{i+1}$).

$$\langle n \rangle_k = (k+1) \langle n-1 \rangle_k + (n-k) \langle n-1 \rangle_{k-1} = \sum_{i=0}^k (-1)^i \binom{n+1}{i} (k-i)!$$

$$\langle 1 \rangle = 1, \quad \langle n \rangle_1 = 2^n - n - 1, \quad \sum_{k=0}^{n-1} \langle n \rangle_k = n!$$

$n k$	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

Modular

- **Fermat:** Se p é primo, então $a^{p-1} \equiv 1 \pmod p$. Se x e m são coprimos e m primo, então $x^k \equiv x^{k \bmod (m-1)} \pmod m$. Euler: $x^{\varphi(m)} \equiv 1 \pmod m$. $\varphi(m)$ é o totiente de Euler.
- **Teorema Chinês do Resto:** Dado um sistema de congruências: $x \equiv a_1 \pmod{m_1}, \dots, x \equiv a_n \pmod{m_n}$ com m_i coprimos dois a dois. E seja $M_i = \frac{m_1 m_2 \dots m_n}{m_i}$ e $N_i = M_i^{-1} \pmod{m_i}$. Então a solução é dada por $x = \sum_{i=1}^n a_i M_i N_i$. Outras soluções são obtidas somando $m_1 m_2 \dots m_n$.

Probabilidade

- **Bertrand Ballot:** Com $p > q$ votos, a probabilidade de sempre haver mais votos do tipo A do que B até o fim é: $\frac{p-q}{p+q}$. Permitindo empates: $\frac{p+1-q}{p+1}$. Multiplicando pela combinação total $\binom{p+q}{q}$, obtém-se o número de possibilidades.
- **Linearidade da Esperança:** $E[aX + bY] = aE[X] + bE[Y]$

- **Variância:** $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - E[X]^2$
- **Uniforme:** $X \in \{a, a + 1, \dots, b\}$, $E[X] = \frac{a+b}{2}$
- **Binomial:** n tentativas com probabilidade p de sucesso: $P(X = x) = \binom{n}{x} p^x (1 - p)^{n-x}$, $E[X] = np$
- **Geométrica:** Número de tentativas até o primeiro sucesso: $P(X = x) = (1 - p)^{x-1} p$, $E[X] = \frac{1}{p}$

9 Extra

9.1 Hash Function

Call

```
g++ hash.cpp -o hash
./hash < code.cpp
```

to get the hash of the code.

The hash ignores comments and whitespaces.
The hash of a line with `}` is the hash of all the code since the `{` that opens it. (is the hash of that context)

(Optional) To make letters upperCase:

```
for(auto&c:s)if('a'<=c) c^=32;
```

```
DE3 string getHash(string s){
E7F   ofstream("z.cpp") << s;
410   system("g++ -E -P -dD -fpreprocessed ./z.cpp | tr -d
      '[:space:]' | md5sum > sh");
F24   ifstream("sh") >> s;
A15   return s.substr(0, 3);
89F }
E8D int main(){
973   string l, t;
C3E   stack<string> st({""});
C61   while(getline(cin, l)){
54F       t = l;
242       for(auto c : l)
A53           if(c == '{') st.push(""); else
733           if(c == '}') t = st.top()+l, st.pop();
1B9           cout << getHash(t) + " " + l << endl;
DB4           st.top() += t;
C14       }
692 }
```